

Experiment #09

Implementation of Parallel Processing using CUDA on GPUs

Objective:

Objective of this lab is to get the knowledge about parallel computation techniques using CUDA.

Software Tools:

- Ubuntu

Theory:

What is CUDA?

CUDA is a parallel computing platform and application programming interface (API) model created by NVidia. When it was first introduced by NVidia, the name CUDA was an acronym for **Compute Unified Device Architecture**, but NVidia subsequently dropped the use of the acronym. CUDA is a platform and programming model for CUDA-enabled GPUs. The platform exposes GPUs for general purpose computing. CUDA provides C/C++ language extension and APIs for programming and managing GPUs.

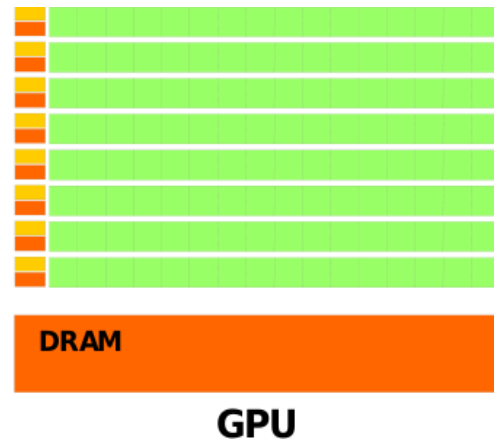
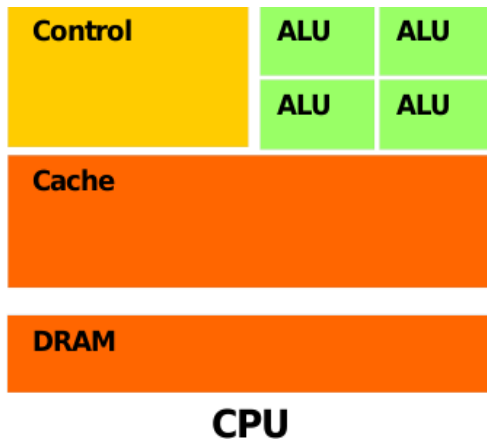
What is GPU?

GPU (Graphics Processing Unit) is a programmable logic chip (processor) specialized for display functions. The GPU renders images, animations and video for the computer's screen. GPUs are located on plug-in cards, in a chipset on the motherboard or in the same chip as the CPU.

Difference between CPU and GPU

CPU	GPU
Really fast caches (great for data reuse)	Lots of math units
Fine branching granularity	Fast access to onboard memory
Lots of different processes/threads	Run a program on each fragment/vertex
CPUs are great for task parallelism	CPUs are great for data parallelism
CPU optimized for high performance on sequential codes (caches and branch prediction)	GPU optimized for higher arithmetic intensity for parallel nature (Floating point operations)

In CUDA programming, both CPUs and GPUs are used for computing. Typically, we refer to CPU and GPU system as *host* and *device*, respectively. CPUs and GPUs are separated platforms with their own memory space. Typically, we run serial workload on CPU and offload parallel computation to GPUs.



A quick comparison between CUDA and C

Following table compares a hello world program in C and CUDA side-by-side.

C	CUDA
<pre>void c_hello() { printf("Hello World!\n"); } int main() { c_hello(); return 0; }</pre>	<pre>__global__ void cuda_hello() { printf("Hello World from GPU!\n"); } int main() { cuda_hello<<<1,1>>>();return 0; }</pre>

The major difference between C and CUDA implementation is `__global__` specifier and `<<<...>>>` syntax. The `__global__` specifier indicates a function that runs on device (GPU). Such function can be called through host code, e.g. the `main()` function in the example, and is also known as "*kernels*".

When a kernel is called, its execution configuration is provided through `<<<...>>>` syntax, e.g; `cuda_hello<<<1,1>>>()`. In CUDA terminology, this is called "*kernel launch*".

For data to be accessible by GPU, it must be presented in the device memory. CUDA provides APIs for allocating device memory and data transfer between host and device memory. Following is the common workflow of CUDA programs.

1. Allocate host memory and initialized host data
2. Allocate device memory
3. Transfer input data from host to device memory
4. Execute kernels
5. Transfer output from device memory to host

Device memory management

CUDA provides several functions for allocating device memory. The most common ones are `cudaMalloc()` and `cudaFree()`. The syntax for both functions are as follow

```
cudaMalloc(void **devPtr, size_t count);  
cudaFree(void *devPtr);
```

`cudaMalloc()` allocates memory of size `count` in the device memory and updates the device pointer `devPtr` to the allocated memory. `cudaFree()` deallocates a region of the device memory where the device pointer `devPtr` points to. They are comparable to `malloc()` and `free()` in C, respectively

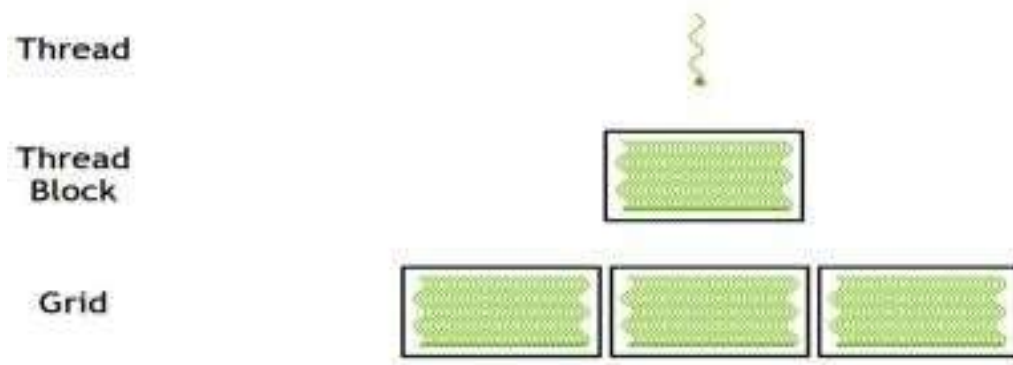
Memory transfer

Transferring data between host and device memory can be done through `cudaMemcpy` function, which is similar to `memcpy` in C. The syntax of `cudaMemcpy` is as follow:

```
cudaMemcpy(void *dst, void *src, size_t count, cudaMemcpyKind kind)
```

This function copy a memory of size `count` from `src` to `dst`. `kind` indicates the direction. For typical usage, the value of `kind` is either `cudaMemcpyHostToDevice` or `cudaMemcpyDeviceToHost`.

Thread hierarchy in CUDA C



CUDA use a kernel execution configuration `<<<...>>>` to tell CUDA runtime how many threads to launch on GPU. CUDA organizes threads into a group called "**thread block**". Kernel can launch multiple thread blocks, organized into a "**grid**" structure.

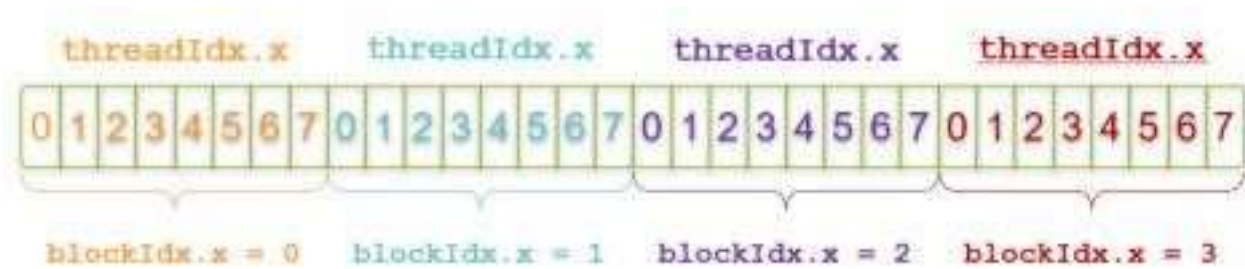
The syntax of kernel execution configuration is as follows

```
<<< M , T >>>
```

Which indicate that a kernel launches with a grid of `M` thread blocks. Each thread block has `T` parallel threads.

CUDA provides built-in variables for accessing thread information. we will use two of them: `threadIdx.x` and `blockIdx.x`.

- threadIdx.x contains the index of the thread within the block
- blockDim.x contains the size of thread block (number of threads in the thread block).



Example Code (.c):

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include "Timer.h"
#define N 10000000
void vector_add(float *out, float *a, float *b, int n) {
    for(int i = 0; i < n; i++){
        out[i] = a[i] + b[i];
    }
}
int main(){
    Timer cpuTime;
    initTimer(&cpuTime, "CPU Time");
    float *a, *b, *out;
    // Allocate memory
    a = (float*)malloc(sizeof(float) * N);
    b = (float*)malloc(sizeof(float) * N);
    out = (float*)malloc(sizeof(float) * N);

    // Initialize array
    for(int i = 0; i < N; i++)
    {
        a[i] = 1.0f;
        b[i] = 2.0f;
    }

    // Main function
    startTimer(&cpuTime);
    vector_add(out, a, b, N);
    stopTimer(&cpuTime);

    // Print time
    printTimer(cpuTime);
}
```

Output:

```
fasih@HCC01:~$ cd lab09
fasih@HCC01:~/lab09$ gcc vector_add.c
fasih@HCC01:~/lab09$ ./a.out
CPU Time = 38.391 msec
fasih@HCC01:~/lab09$
```

Example Code (.cu):

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <cuda.h>
#include "Timer.h"
#define N 10000000
__global__ void vector_add(float *out, float *a, float *b, int n) {
    for(int i = 0; i < n; i++){
        out[i] = a[i] + b[i];
    }
}

int main(){
    Timer kernelTime; initTimer(&kernelTime, "Kernel Time");
    float *a, *b, *out;
    float *d_a, *d_b, *d_out;

    // Allocate host memory
    a = (float*)malloc(sizeof(float) * N);
    b = (float*)malloc(sizeof(float) * N);
    out = (float*)malloc(sizeof(float) * N);

    // Initialize host arrays
    for(int i = 0; i < N; i++){
        a[i] = 1.0f;
        b[i] = 2.0f;
    }

    // Allocate device memory
    cudaMalloc((void**)&d_a, sizeof(float) * N);
    cudaMalloc((void**)&d_b, sizeof(float) * N);
    cudaMalloc((void**)&d_out, sizeof(float) * N);

    // Transfer data from host to device memory
    cudaMemcpy(d_a, a, sizeof(float) * N, cudaMemcpyHostToDevice);
    cudaMemcpy(d_b, b, sizeof(float) * N, cudaMemcpyHostToDevice);

    // Executing kernel
    startTimer(&kernelTime);
```

```

vector_add<<<1,1>>>(d_out, d_a, d_b, N);
stopTimer(&kernelTime);

// Transfer data back to host memory
cudaMemcpy(out, d_out, sizeof(float) * N,
cudaMemcpyDeviceToHost);

printf("PASSED\n");

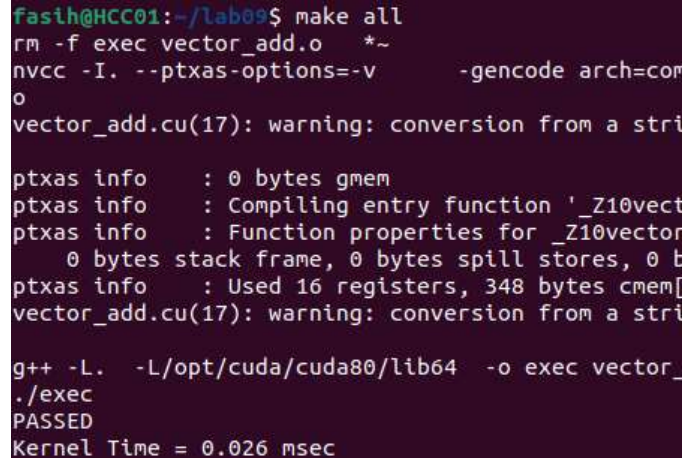
// Print time
printTimer(kernelTime);

// Deallocate device memory
cudaFree(d_a);
cudaFree(d_b);
cudaFree(d_out);

// Deallocate host memory
free(a);
free(b);
free(out);
}

```

Output:



```

fasth@HCC01:~/lab09$ make all
rm -f exec vector_add.o *~
nvcc -I. --ptxas-options=-v -gencode arch=compute_30
vector_add.cu(17): warning: conversion from a string to 'void*' is not safe
ptxas info      : 0 bytes gmem
ptxas info      : Compiling entry function '_Z10vector_add'
ptxas info      : Function properties for _Z10vector_add
      0 bytes stack frame, 0 bytes spill stores, 0 bytes
ptxas info      : Used 16 registers, 348 bytes cmem[0]
vector_add.cu(17): warning: conversion from a string to 'void*' is not safe
g++ -L. -L/opt/cuda/cuda80/lib64 -o exec vector_add.o
./exec
PASSED
Kernel Time = 0.026 msec

```

Example Code using Threads (.cu):

```

#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <cuda.h>
#include "Timer.h"
#define N 10000000
__global__ void vector_add(float *out, float *a, float *b, int n) {
    int index = threadIdx.x;
    int stride = blockDim.x;

```

```

    for(int i = index; i < n; i += stride){
        out[i] = a[i] + b[i];
    }
}
int main(){
    Timer kernelTime;
    initTimer(&kernelTime, "Kernel Time");
    float *a, *b, *out;
    float *d_a, *d_b, *d_out;

    // Allocate host memory
    a = (float*)malloc(sizeof(float) * N);
    b = (float*)malloc(sizeof(float) * N);
    out = (float*)malloc(sizeof(float) * N);

    // Initialize host arrays
    for(int i = 0; i < N; i++){
        a[i] = 1.0f;

        b[i] = 2.0f;
    }

    // Allocate device memory
    cudaMalloc((void**)&d_a, sizeof(float) * N);
    cudaMalloc((void**)&d_b, sizeof(float) * N);
    cudaMalloc((void**)&d_out, sizeof(float) * N);

    // Transfer data from host to device memory
    cudaMemcpy(d_a, a, sizeof(float) * N, cudaMemcpyHostToDevice);
    cudaMemcpy(d_b, b, sizeof(float) * N, cudaMemcpyHostToDevice);

    // Executing kernel
    startTimer(&kernelTime);
    vector_add<<<1,256>>>>(d_out, d_a, d_b, N);
    stopTimer(&kernelTime);

    // Transfer data back to host memory
    cudaMemcpy(out, d_out, sizeof(float) * N, cudaMemcpyDeviceToHost);

    printf("PASSED\n");

    // Print time
    printTimer(kernelTime);

    // Deallocate device memory
    cudaFree(d_a);
    cudaFree(d_b);
    cudaFree(d_out);
}

```

```
// Deallocate host memory
free(a);
free(b);
free(out);
}
```

Output:

```
fasih@HCC01:~/lab09$ make all
rm -f exec vector_add_thread.o *~
nvcc -I. --ptxas-options=-v -gencode arch=compute_30
or_add_thread.o
vector_add_thread.cu(20): warning: conversion from
ptxas info      : 0 bytes gmem
ptxas info      : Compiling entry function '_Z10vector
ptxas info      : Function properties for _Z10vector
0 bytes stack frame, 0 bytes spill stores, 0 b
ptxas info      : Used 9 registers, 348 bytes cmem[0]
vector_add_thread.cu(20): warning: conversion from
g++ -L. -L/opt/cuda/cuda80/lib64 -o exec vector_
./exec
PASSED
Kernel Time = 0.018 msec
fasih@HCC01:~/lab09$
```


Lab Task:

Write a code for the following using CUDA.

- Write a Program for the addition of two vectors in serial and parallel functions. Also find the execution time taken by CUDA Kernel and C Function.
 - Write a program to find the dot product of two vectors in serial function and parallel function. Print both execution time and Compare results.
-